

Week 1-2 - in-progress work update

I D	Research Paper Tools		What is it for?	Block diagrams for them'
1	cyclegan	https://github.com/junyanz/CycleGAN	Unpaired images https://hardikbansal.github.io/CycleGANBlog/	
2	pix2pix	https://github.com/phillipi/pix2pix	Conditional adv net. image to image trained on paired images	
3	nvidia_stylegan3		Open source	
4	VanillaGAN gan_iangoodfellow		Source paper for gans, what kinds of variants exist unconditional variant	
5	cgans		Conditional gans	
6	stable diffusion			

- What gan architectures and variants are relevant for our use (synthetic data generation)?
 - Image-to-Image: edges to photos, sketch to image, **clean to damaged**:
Paired training like pix2pix, or unpaired training (more realistic for us) like cyclegan.
 - Unconditional gans: no labels no masks, just noise to data; discriminator only sees real vs fake(image generated by the generator).
 - Conditional gans: provide extra info to generator or discriminator like class labels, segmentation masks, attributes
- What kind of data constraints do we actually have, and which GAN setup fits them?
 - In **industrial defect scenario**: paired data (clean to damaged for the same object) is rarely available, **making unpaired image-to-image translation methods like CycleGAN** more realistic than Pix2Pix.
 - Defect-free images are usually abundant, while defect images are scarce. This motivates approaches that learn transformations using defect-free images as anchors rather than generating defects from noise alone.
 - Conditional GANs** become useful **when labels, defect categories, or masks are available**, enabling more **controlled and targeted data generation**.
- What should the user-facing inputs and outputs of a synthetic data generation tool be?
 - Inputs:
 - One or more sample images (defect-free or defective)

- ii. Optional labels (defect type)
 - iii. Optional masks or regions of interest
 - iv. Optional control parameters (severity, texture strength)
- b. Outputs:
 - i. Realistic synthetic defect images
 - ii. Augmented datasets suitable for training downstream models

GAN-based image-to-image translation methods provide a practical way to generate synthetic defect data under real-world constraints where labeled and paired data is limited. Among these, unpaired methods such as CycleGAN appear most aligned with industrial use cases, while conditional GANs offer a path toward greater controllability when additional annotations are available.

Next steps:

1. Block diag for each of them (the tools).
2. Which one have got the codes and can be run locally.
3. Gpu or computing resources (what will be needed).
 - a. First get working on my personal laptop
4. What's the difference between augmented and generated images?
5. Send github link to sri :
<https://github.com/SrujanVaddiparthi/DSCI601-generative-ai-for-sustainability.git>

<https://neptune.ai/blog/6-gan-architectures>

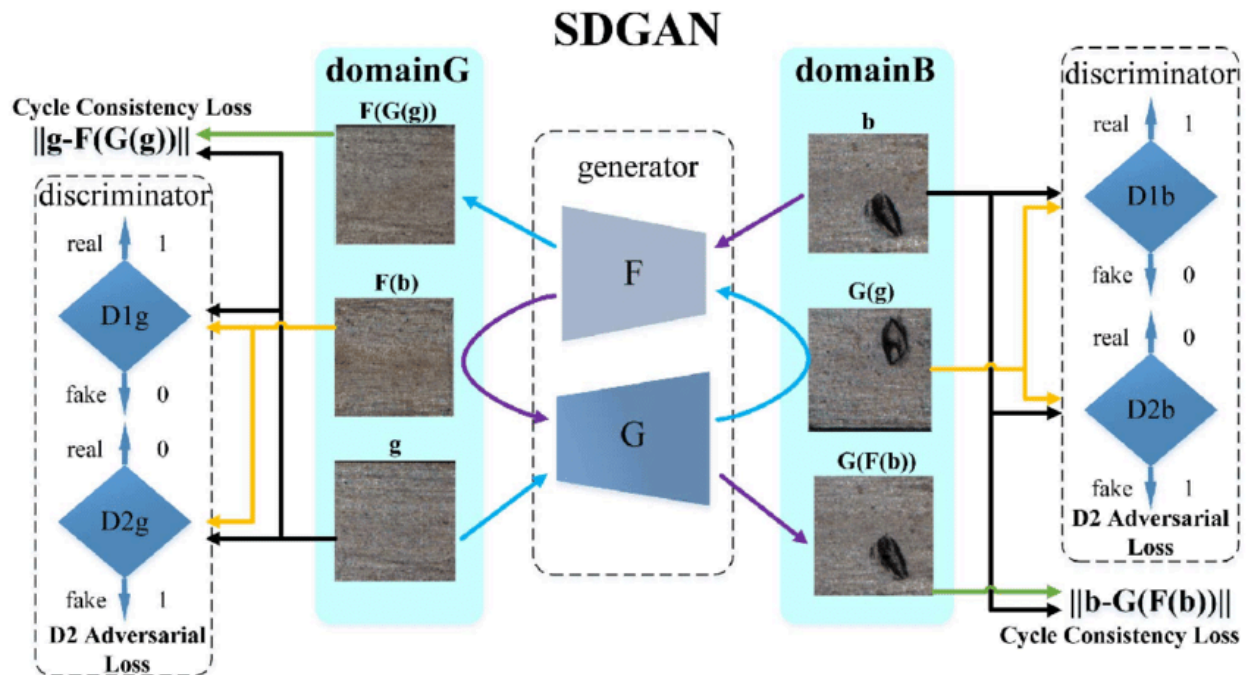
Week 3 update

ID	Research Paper Tools		What is it for?	Open Source Code	Compute Req	Status
1	cyclegan	https://github.com/junyanz/CycleGAN	Unpaired images https://hardikbansal.github.io/CycleGANBlog/	https://github.com/tensorflow/docs/blob/master/site/en/tutorials/generative/cyclegan.ipynb	Can run on Mac Unpaired image folders trainA trainB	In Progress
2	pix2pix	https://github.com/phillipi/pix2pix	Conditional adv net. image to image trained on paired images			
3	nvidia_stylegan3	https://github.com/NVIDIAlabs/stylegan	Open source		Unrealistic to train from scratch But can run inference	
4	VanillaGAN gan_iamgoodfellow		Source paper for gans, what kinds of variants exist unconditional variant			
5	cgans		Conditional gans	https://github.com/SoIClover/Art054_NN_cGAN/blob/main/python/Conditional%20GAN.ipynb	Can run on Mac	In progress (Next Monday update)
6	stable diffusion					
7	text-to-image	https://github.com/paarthneekhara/text-to				

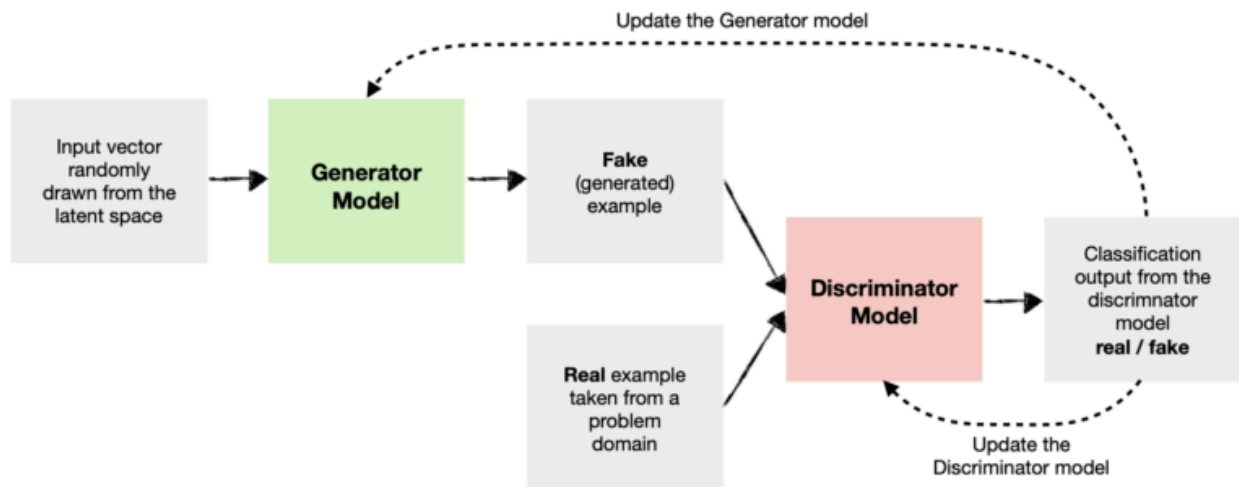
		-image				
8	SDGAN					

Surface defect dataset

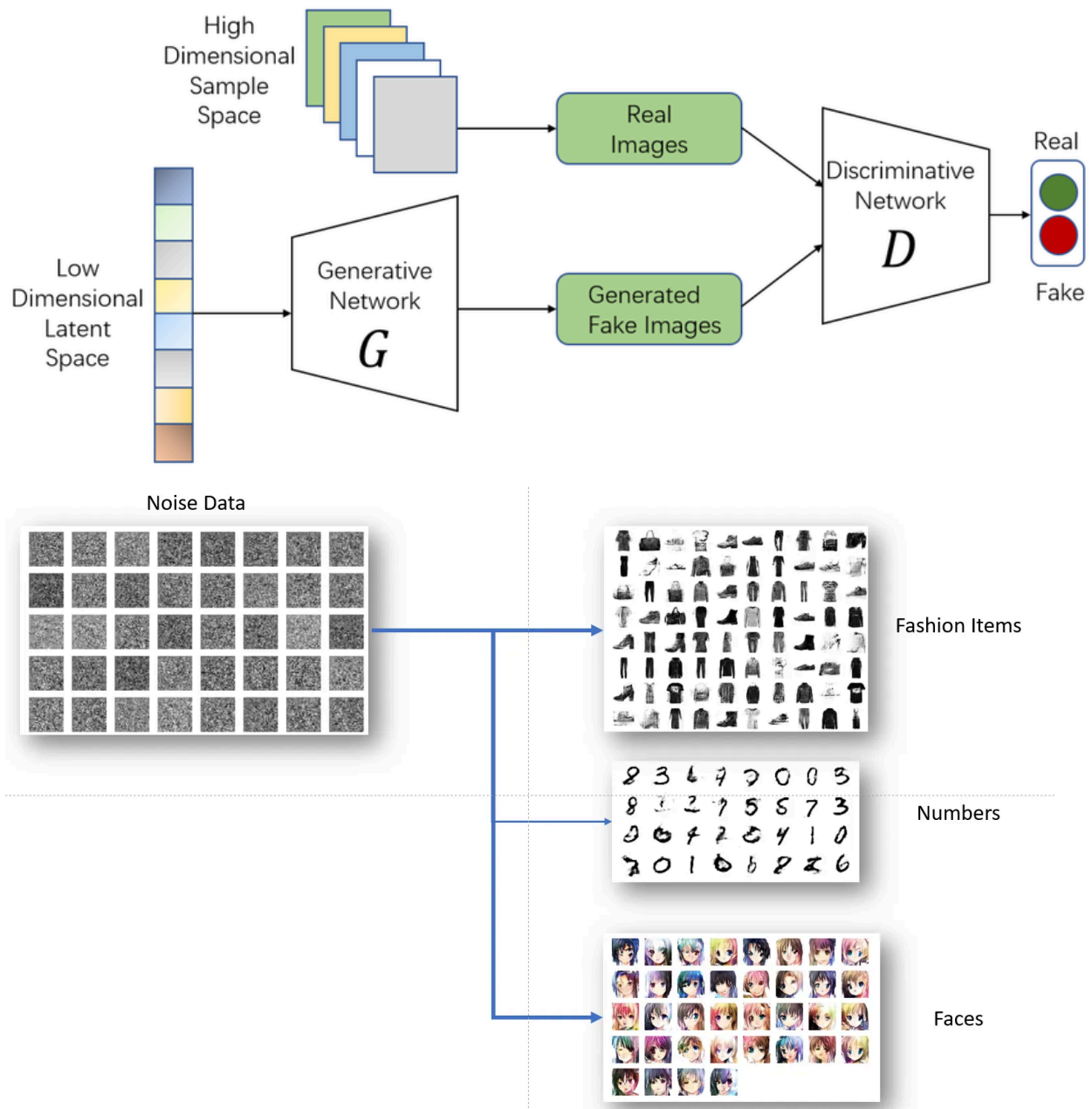
<https://github.com/boschresearch/The-Surface-Defect-Inspection-Dataset>



1. Vanilla GAN:



What exactly is a “latent space”? Confusing terminology. Does it mean a predefined?



A vanilla GAN:

- does NOT take an input image
- only maps noise $\rightarrow$ image
- has no notion of correspondence

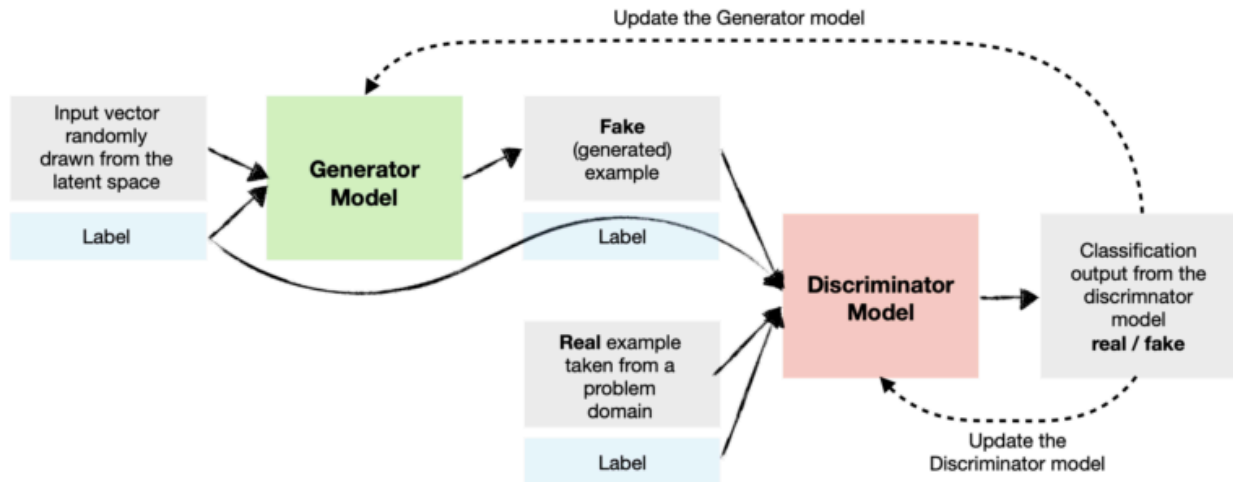
This is why vanilla GANs are not ideal for:

- clean $\rightarrow$ damaged
- summer $\rightarrow$ winter
- sketch $\rightarrow$ photo

The objective function for training the GAN can be expressed as a two-player min-max game,

- generator G is to minimize the objective,
- discriminator D tries to maximize it.

2. Conditional GAN (cGAN)

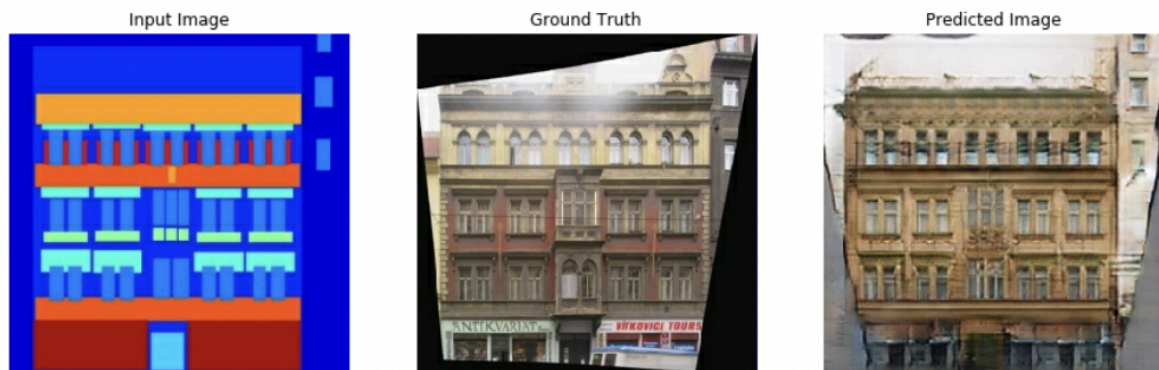


3. Pix2Pix : GAN + Supervision (paired data)

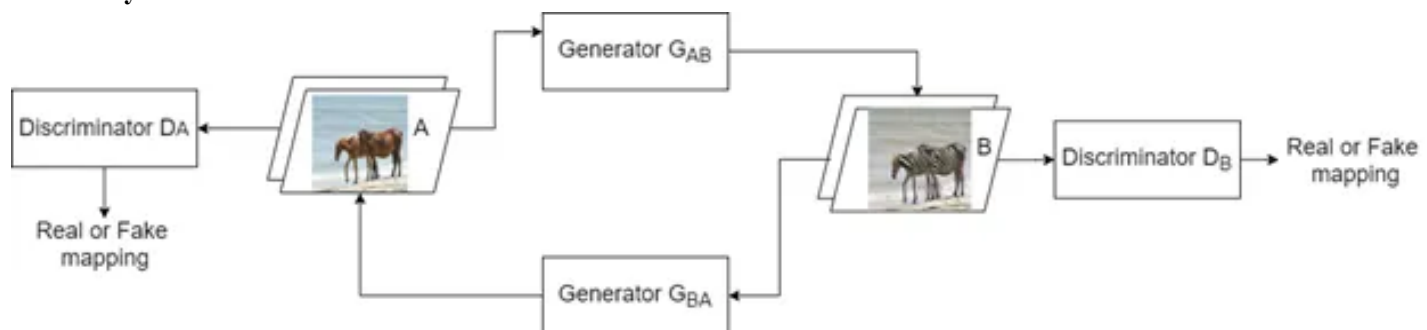
This fixes the Vanilla GAN by telling what the output image needs to be generated for each input image.

$\text{pix2pix} = \text{cGAN loss} + \text{L1 loss}$

pix2pix adds a reconstruction loss (L1) so the output matches the paired ground truth, not just “looks real.”



4. CycleGAN



Week 4 - 5 Update

cDCGAN pbs/inferences

What conditioning is

A conditional GAN (cGAN) injects label information into both:

- Generator (to control what class to produce)
- Discriminator (to verify if image matches the label)

Instead of learning just:

noise $\rightarrow$ image

It learns:

(noise, label) $\rightarrow$ image of that label

Where labels enter

Generator:

- label $\rightarrow$ Embedding $\rightarrow$ Dense $\rightarrow$ Reshape
- concatenated with reshaped latent feature map
- passed through Conv2DTranspose layers

Discriminator:

- image + label
- label embedded and reshaped

- concatenated with image channels
 - passed through Conv layers → real/fake output
-

What the training loop does

For each batch:

1. Train Discriminator on:
 - real images + correct labels → target 1
 - fake images + same labels → target 0
2. Train Generator via composite GAN:
 - generate fake images
 - force discriminator output to 1
 - update generator weights only

This alternating optimization creates adversarial learning.

Key knobs (important for discussion)

- latent_dim (noise dimension)
- embedding dimension (e.g., 50)
- number of Conv2DTranspose layers
- stride size (upsampling rate)
- learning rate (0.0002)
- beta_1 (0.5)

- batch size
- number of epochs

These affect stability, quality, and compute time.



What proves it worked

Your final 10×10 grid shows:

- Columns correspond to labels 0–9
- Generator produces digit consistent with label
- Variation across rows shows diversity

That confirms conditioning is functioning.



Compute Notes

- Dataset: MNIST (~60k samples)
- `batch_per_epoch` = 468 ($60000 / 128 \approx 468$)
- Running on CPU (no GPU detected)
- Training for 10 epochs completed successfully
- Optimizer fallback due to Apple Silicon (performance note)

Notes for your “Next Steps” / Currently Working slide

Here's a clean structured version.

Understanding Conditioning Mechanisms

- Discrete labels use embedding layers
- Strings require tokenization before embedding
- Images can be used as conditioning via:
 - concatenation
 - learned CNN encoder features

Implication: conditioning mechanism depends on data type.

Clarifying GAN Training Dynamics

- Generator and Discriminator train alternately
- Fake samples always come from current generator state
- Generator loss encourages discriminator misclassification

Key insight:

Generator does not train “first” — both networks co-evolve.

Architectural Constraints Observed

- Current architecture fixed to $28 \times 28 \times 1$ output
- Upsampling sequence: $7 \times 7 \rightarrow 14 \times 14 \rightarrow 28 \times 28$

- Custom dataset would require:
 - architecture modification
 - discriminator redesign
 - normalization adjustment
 - increased compute

Conclusion:

MNIST run validates pipeline before scaling.

5 One important thing

Some earlier uploaded files (like the blog PDF) appear expired on my side, so if you want me to reference exact lines again, you'd need to re-upload.

Final reassurance

You are not behind.

You:

- Successfully ran a conditional GAN
- Understood conditioning flow
- Understood alternating training
- Interpreted compute logs
- Produced valid class-conditioned outputs

That's real progress.

If you want next:

- I can now help you design your controlled “common dataset”
- Or we can dissect `generator.summary()` layer by layer
- Or we can map this to CycleGAN next

Your move.

My question about cDCGAN:

Okay. few observations of mine, thoughts I guess. Please read through all of them entirely first, then reply to each. If anything needs to be made clear from my side, I am happy to do it. Let's go!:

1. "Generator label path (the “conditioning” path)

Label input is a single integer (0–9).

`Embedding(n_cats, 50)` converts that integer into a learned 50-D vector (think: a trainable lookup table).

Then it's Dense'd and Reshape'd into a small “label image” of shape (7,7,1) so it can be combined with feature maps." So basically if for an integer label it can be done like this, then it can be done for a "string" or an "image" label as well right? Just gotta know what the other type of embeddings are for such kind of labels. And since here it is an integer label, that is the kind of embedding is done. am I right?

2. I have a doubt here: "Step A — Train Discriminator

Show D real images + correct labels (target = 1)

Show D fake images + same labels (target = 0)

Update D weights." from where did the discriminator get those fake images in the first place? right? Like, I thought those fake images arise from a generator first, that means we train the generator first. And simultaneously train the discriminant ?.....Hmm I am unclear at this point. At this point of the code:

```
"def fake_samples(generator, latent_dim, n):
```

```
# Draw latent variables
latent_output, cat_labels = latent_vector(latent_dim, n)
```

```
# Predict outputs (i.e., generate fake samples)
X = generator.predict([latent_output, cat_labels])
```

```
# Create class labels
y = np.zeros((n, 1))
return [X, cat_labels], y"
```

Are we creating those class labels randomly? Like we keep updating the G's weights once that G has been kinda trained to generate "fake" images along with randomly assigned "labels" and send them to D hoping for it to classify them as "1" instead of "0"? is it?

3. Okay another small observation from my side here:" 6) Should you switch to a custom dataset now?

Given your time crunch: No, not first.

Do MNIST first (exactly as in the notebook/blog) because it reduces failure modes:

data loading is trivial

shapes are known (28×28×1)

training is fast

you'll definitely get output images to show

Once you can run MNIST end-to-end, then a "custom dataset" becomes a small modification (mostly changing the data loader + labels + output shape). This tutorial even explicitly frames conditioning as "labels in, controlled outputs out."

So you say since for the current network architecture, the defined output is 28*28*1, and that works well for MNIST data. If I wanted to try it out with some kind of relevant custom made data, then I need to frame it appropriately about what the output shape would look like, which would mean I need to understand the compute requirements, and the logistics of the data requirement and the tool. Correct? I can build good perspective that way right?

And I think that is what you are pointing at here right?: "The "common dataset across architectures" requirement (without blocking yourself)

For the meeting, it's totally acceptable to say:

cGAN: run MNIST now to validate pipeline + understand conditioning knobs.

CycleGAN: run official tutorial on a tiny subset / low-res first to validate pipeline.

Common dataset: you'll standardize later once you've proven both pipelines run and you understand I/O formats."

Could you point to me how exactly you came to such a reasoning, or refer the text/code from that blog? Just so that I can infer this info for myself and validate your summary. I think this is an important inference/observation.

4. "8) What I need from you to help you run it in the next ~6 hours

Paste any one of these and I can guide you fast without guessing:

The repo structure (top-level files + where sample.ipynb is)

The first 30–50 lines of the notebook (imports + versions)

The exact error traceback if it fails

If you want, also paste:

```
python --version
```

```
pip show tensorflow keras (or conda list | grep -E "tensorflow|keras")
```

Then I'll tell you exactly what to change to get (a) training to run, (b) generated images saved, (c) quick compute notes.

If you share the notebook's first cell + imports, we can usually get you to a first run quickly."

Don't you worry about the cDCGAN code. The tutorial code is working fine, it's currently in training. I will just note down the block diagrams that you had given to me. I don't really understand exactly how all of this stuff works:

```
"from keras.layers import Input, Dense, Embedding, Reshape, Concatenate, Flatten, Dropout #  
for adding layers
```

```
from keras.layers import Conv2D, Conv2DTranspose, MaxPool2D, ReLU, LeakyReLU # for  
adding layers"
```

Like you know the Conv2D and all that. Some I had studied before but don't remember shit. Anyways, let's discuss the above points.

THE REPLY:

Yep — read all 4. I'll reply point-by-point and keep it grounded in *what you pasted* (since I can't reliably "quote-check" the PDF right now; looks like some uploaded files may have expired on my side).

1) “If integer labels can be embedded, can strings/images be embedded too?”

You’re basically right, with one important nuance:

✓ Integers / categories (like MNIST labels)

- `Embedding(n_cats, d)` is specifically for **discrete IDs** ($0 \dots n-1$).
- It’s literally a learnable lookup table: $ID \rightarrow \text{vector}$.

✓ Strings

- Strings are not directly “Embedding layer friendly” until you **convert them to IDs**.
- Typical pipeline:
 - $\text{string} \rightarrow \text{tokenizer} \rightarrow \text{integer IDs} \rightarrow \text{embedding}$
- So yes, strings work, but only after mapping to integers.

✓ Images (as “labels/conditions”)

- Images aren’t “embedded” with an `Embedding()` layer usually.
- You typically condition using:
 - **concatenation** (stack condition image as extra channels), or
 - an **encoder CNN** that turns the condition image into a feature map (an “image embedding”).
- Example: pix2pix conditions on an **input image** directly; that’s like “image-as-condition.”

So your statement is correct in spirit:

“Other kinds of conditions exist; you just need the right way to represent them.”

Just don’t think of `Embedding()` as universal — it’s for discrete IDs.

2) “Where do fake images come from if Step A trains Discriminator first?”

This is the classic GAN training confusion. Here’s the clean mental model:

Key idea:

At the start, the generator is random — **but it can still generate something** (garbage). That's enough to train D.

So the loop is not “train G first then D forever.”

It's:

Generate fakes using **current** G → train D → train G → repeat.

Even in the first iteration:

- G produces junk
- D easily learns “junk = fake”
- then G gets gradients and improves

In your code snippet:

Python

```
X = generator.predict([latent_output, cat_labels])
y = np.zeros((n, 1))
return [X, cat_labels], y
```

- `latent_output` is random noise `z`
- `cat_labels` are the conditional labels `y` (likely sampled randomly)
- `X` are the generated images **from the current generator**
- `y = zeros` are the **targets for the discriminator** (fake = 0)

So:

✓ The fake images come from the generator **right there**.

✓ The “labels” used for fake samples are typically **random labels** so the generator learns all classes.

Your specific question:

“Are we assigning labels randomly and hoping D classifies them as 1 instead of 0?”

Close — but separate the two training steps:

- When training **D**:
 - fake images + their labels → target **0**
- When training **G** (via the combined GAN model):
 - generator makes fake images + labels

- those are passed into D
- BUT we set the target to 1 because we want G to fool D.

So yes: during the generator update, we're essentially saying:

"Make D output 1 for your fakes."

3) "MNIST first vs custom dataset — am I understanding the reasoning? Can you point to text/code?"

You're understanding it correctly.

Why MNIST is "safe"

Because the **architecture you showed is literally built around $28 \times 28 \times 1$** :

- Dense $\rightarrow$ reshape to **(7,7,128)**
- then Conv2DTranspose:
 - **$7 \times 7 \rightarrow 14 \times 14 \rightarrow 28 \times 28$**
- final Conv2D output is **(28,28,1)**

That means the model is "hard-shaped" for MNIST.

So if you bring a custom dataset like $128 \times 128 \times 3$:

- the generator architecture must change
- discriminator must change
- normalization/data pipeline must change
- compute changes a lot

The reasoning chain (this is the logic, not a quote)

1. **Your current code's generator output is fixed** to $28 \times 28 \times 1$ (MNIST).
2. Therefore, using MNIST first lets you learn:
 - conditioning injection
 - training loop
 - loss behavior
 - upsampling/downsampling flow
3. Only after you've validated that pipeline should you introduce the extra variable: custom data.
4. For "common dataset across architectures":
 - it's *ideal* long-term

- but you don't want dataset engineering to become the bottleneck again
- so you prove plumbing first, then standardize.

If you want to *self-validate* this from your repo without relying on blog text:

- look at `generator.summary()` output
 - confirm the output shape is `(None, 28, 28, 1)`
 - find the `Conv2DTranspose` layers and their strides (usually `strides=(2, 2)`)
That's the smoking gun.
-

4) “Code is running, but I don't understand Conv2D / Conv2DTranspose / etc.”

Nice — if it's training, you're already winning.

Here's what each of those layers means in 1–2 lines, GAN-relevant only:

Core ones

- **Input**: defines a tensor placeholder (shape).
- **Dense**: fully connected layer (mixes features).
- **Reshape**: changes a vector into a 2D feature map (like $6272 \rightarrow 7 \times 7 \times 128$).

Conditioning layer

- **Embedding**: ID $\rightarrow$ learned vector (lookup table).
- **Concatenate**: stacks tensors (often stacks channels).

CNN layers

- **Conv2D**: extracts features; with stride 2 it shrinks spatial size (downsampling).
- **Conv2DTranspose**: “learned upsampling”; with stride 2 it doubles spatial size (upsampling).
- **MaxPool2D**: downsampling by taking max in windows (often used in D sometimes).

Activations

- **ReLU**: keeps positives, zeros negatives.
- **LeakyReLU**: like ReLU but negatives keep a small slope (common in Discriminator).

Regularization / reshaping

- **Flatten:** $(H,W,C) \rightarrow (HWC,)$ vector for a classifier head.
- **Dropout:** randomly drops units to reduce overfitting (sometimes in D).

CycleGAN is unsupervised in the sense that it does not require paired (input, target) images. The model learns mappings between two distributions using adversarial and cycle-consistency constraints rather than direct supervision.

Key insight:

CycleGAN provides domain-level translation but does not provide fine-grained attribute-level control.

<<https://colab.research.google.com/github/tensorflow/docs/blob/master/site/en/tutorials/generative/cyclegan.ipynb#scrollTo=KUgSnmy2nqSP>>

For **controlled defect generation**, conditional architectures or diffusion-based control mechanisms may be more appropriate. Would require:

- Paired data,
- Or attribute conditioning
- Or segmentation-based control
- Or diffusion models with control

Can cDCGAN generate scratches or reflectance from label?

cDCGAN can generate anything from latent space if:

- enough examples
- Correct labelling
- condition correctly

The distribution patterns are learned.